

NAME:UMER FAROOQ

SAP: 56154

SECTION:SE5-2

SUBJECT:MAD

QUIZ 3

```
import 'package:flutter/material.dart';

void main() {
  runApp(const FlashcardApp());
}

class FlashcardApp extends StatelessWidget {
  const FlashcardApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flashcard Quiz',
      theme: ThemeData(
        primarySwatch: Colors.deepPurple,
        scaffoldBackgroundColor: Colors.grey[100],
      ),
      home: const FlashcardHomePage(),
    );
  }
}

class Flashcard {
  final String id;
  final String question;
  final String answer;
  bool isLearned;

  Flashcard({
    required this.id,
    required this.question,
    required this.answer,
    this.isLearned = false,
  });
}

class FlashcardHomePage extends StatefulWidget {
  const FlashcardHomePage({Key? key}) : super(key: key);

  @override
  State<FlashcardHomePage> createState() => _FlashcardHomePageState();
}
```

```

class _FlashcardHomePageState extends State<FlashcardHomePage> {
  final GlobalKey<AnimatedListState> _listKey =
GlobalKey<AnimatedListState>();
  List<Flashcard> _flashcards = [];
  int _learnedCount = 0;

  @override
  void initState() {
    super.initState();
    _initializeFlashcards();
  }

  void _initializeFlashcards() {
    _flashcards = [
      Flashcard(
        id: '1',
        question: 'What is Flutter?',
        answer: 'Flutter is Google\'s UI toolkit for building natively
compiled applications for mobile, web, and desktop from a single codebase.',
      ),
      Flashcard(
        id: '2',
        question: 'What is a Widget in Flutter?',
        answer: 'A Widget is the basic building block of a Flutter UI.
Everything in Flutter is a widget, from buttons to padding to layouts.',
      ),
      Flashcard(
        id: '3',
        question: 'What is StatefulWidget?',
        answer: 'A StatefulWidget is a widget that has mutable state. It can
rebuild when its state changes using setState().',
      ),
      Flashcard(
        id: '4',
        question: 'What is Dart?',
        answer: 'Dart is the programming language used to write Flutter apps.
It\'s an object-oriented, class-based language with C-style syntax.',
      ),
      Flashcard(
        id: '5',
        question: 'What is a BuildContext?',
        answer: 'BuildContext is a handle to the location of a widget in the
widget tree. It\'s used to access theme data, navigate, and more.',

```

```

    ),
  ];
  _learnedCount = 0;
}

Future<void> _refreshFlashcards() async {
  // Simulate network delay
  await Future.delayed(const Duration(seconds: 1));

  setState(() {
    _initializeFlashcards();
  });

  // Rebuild the animated list
  _listKey.currentState?.setState(() {});
}

void _markAsLearned(int index) {
  final card = _flashcards[index];

  setState(() {
    card.isLearned = true;
    _learnedCount++;
    _flashcards.removeAt(index);
  });

  _listKey.currentState?.removeItem(
    index,
    (context, animation) => _buildFlashcardItem(card, animation, index,
false),
    duration: const Duration(milliseconds: 300),
  );
}

void _addNewFlashcard() {
  final newCard = Flashcard(
    id: DateTime.now().toString(),
    question: 'What is Hot Reload?',
    answer: 'Hot Reload allows you to quickly see changes in your code
without restarting the app or losing its state.',
  );

  setState(() {

```

```

        _flashcards.insert(0, newCard);
    });

    _listKey.currentState?.insertItem(0, duration: const
Duration(milliseconds: 400));

    ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
            content: Text('New flashcard added!'),
            duration: Duration(seconds: 2),
        ),
    );
}

Widget _buildFlashcardItem(Flashcard card, Animation<double> animation, int
index, bool isDismissible) {
    return SizeTransition(
        sizeFactor: animation,
        child: isDismissible
            ? Dismissible(
                key: Key(card.id),
                direction: DismissDirection.horizontal,
                onDismissed: (direction) {
                    _markAsLearned(index);
                    ScaffoldMessenger.of(context).showSnackBar(
                        SnackBar(
                            content: Text('Marked "${card.question}" as learned!'),
                            duration: const Duration(seconds: 2),
                            backgroundColor: Colors.green,
                        ),
                    );
                },
                background: Container(
                    margin: const EdgeInsets.symmetric(horizontal: 16, vertical:
8),
                    decoration: BoxDecoration(
                        color: Colors.green,
                        borderRadius: BorderRadius.circular(12),
                    ),
                    alignment: Alignment.centerLeft,
                    padding: const EdgeInsets.only(left: 20),
                    child: const Icon(Icons.check_circle, color: Colors.white,
size: 32),

```

```

    ),
    secondaryBackground: Container(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical:
8),
      decoration: BoxDecoration(
        color: Colors.green,
        borderRadius: BorderRadius.circular(12),
      ),
      alignment: Alignment.centerRight,
      padding: const EdgeInsets.only(right: 20),
      child: const Icon(Icons.check_circle, color: Colors.white,
size: 32),
    ),
    child: FlashcardWidget(card: card),
  )
  : FlashcardWidget(card: card),
);
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: CustomScrollView(
      slivers: [
        SliverAppBar(
          expandedHeight: 120,
          floating: false,
          pinned: true,
          backgroundColor: Colors.deepPurple,
          flexibleSpace: FlexibleSpaceBar(
            title: Text(
              'Progress: $_learnedCount of ${_flashcards.length +
_learnedCount} learned',
              style: const TextStyle(fontSize: 14),
            ),
          ),
          background: Container(
            decoration: BoxDecoration(
              gradient: LinearGradient(
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
                colors: [
                  Colors.deepPurple,
                  Colors.deepPurple[300]!,

```

```

    ],
  ),
),
child: Center(
  child: Padding(
    padding: const EdgeInsets.only(top: 40),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Icon(Icons.school, size: 40, color:
Colors.white),

        const SizedBox(height: 8),
        if (_flashcards.isNotEmpty)
          LinearProgressIndicator(
            value: _learnedCount / (_flashcards.length +
_learnedCount),

            backgroundColor: Colors.white30,
            valueColor: const
AlwaysStoppedAnimation<Color>(Colors.white),

          ),
        ],
      ),
    ),
  ),
),
),
),
),
),
),
),
),
SliverToBoxAdapter(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Text(
      _flashcards.isEmpty
        ? '🎓 All cards learned! Pull down to refresh.'
        : 'Swipe cards to mark as learned',
      style: TextStyle(
        fontSize: 16,
        color: Colors.grey[700],
        fontWeight: FontWeight.w500,
      ),
      textAlign: TextAlign.center,
    ),
  ),
),
),
),
),

```

```

RefreshIndicator(
  onRefresh: _refreshFlashcards,
  child: AnimatedList(
    key: _listKey,
    shrinkWrap: true,
    physics: const NeverScrollableScrollPhysics(),
    initialItemCount: _flashcards.length,
    itemBuilder: (context, index, animation) {
      if (index >= _flashcards.length) return const
SizedBox.shrink();
      return _buildFlashcardItem(_flashcards[index], animation,
index, true);
    },
  ),
),
],
),
floatingActionButton: FloatingActionButton.extended(
  onPressed: _addNewFlashcard,
  icon: const Icon(Icons.add),
  label: const Text('Add Card'),
  backgroundColor: Colors.deepPurple,
),
);
}
}

class FlashcardWidget extends StatefulWidget {
  final Flashcard card;

  const FlashcardWidget({Key? key, required this.card}) : super(key: key);

  @override
  State<FlashcardWidget> createState() => _FlashcardWidgetState();
}

class _FlashcardWidgetState extends State<FlashcardWidget> {
  bool _isExpanded = false;

  void _toggleCard() {
    setState(() {
      _isExpanded = !_isExpanded;
    });
  }
}

```



```

}

@override
Widget build(BuildContext context) {
  return GestureDetector(
    onTap: _toggleCard,
    child: AnimatedContainer(
      duration: const Duration(milliseconds: 300),
      curve: Curves.easeInOut,
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.1),
            blurRadius: 10,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 8,
vertical: 4),
                decoration: BoxDecoration(
                  color: Colors.deepPurple.withOpacity(0.1),
                  borderRadius: BorderRadius.circular(4),
                ),
                child: const Text(
                  'Q',
                  style: TextStyle(
                    color: Colors.deepPurple,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
              const SizedBox(width: 12),
            ],
          ),
        ],
      ),
    ),
  );
}

```

```

Expanded(
  child: Text(
    widget.card.question,
    style: const TextStyle(
      fontSize: 18,
      fontWeight: FontWeight.w600,
      color: Colors.black87,
    ),
  ),
),
Icon(
  _isExpanded ? Icons.expand_less : Icons.expand_more,
  color: Colors.deepPurple,
),
],
),
AnimatedCrossFade(
  firstChild: const SizedBox.shrink(),
  secondChild: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const SizedBox(height: 16),
      const Divider(),
      const SizedBox(height: 16),
      Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 8,
vertical: 4),

            decoration: BoxDecoration(
              color: Colors.green.withOpacity(0.1),
              borderRadius: BorderRadius.circular(4),
            ),
            child: const Text(
              'A',
              style: TextStyle(
                color: Colors.green,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          const SizedBox(width: 12),

```

```

Expanded(
  child: Text(
    widget.card.answer,
    style: TextStyle(
      fontSize: 16,
      color: Colors.grey[800],
      height: 1.5,
    ),
  ),
),
],
),
const SizedBox(height: 12),
Text(
  '💡 Swipe to mark as learned',
  style: TextStyle(
    fontSize: 12,
    color: Colors.grey[500],
    fontStyle: FontStyle.italic,
  ),
),
],
),
crossFadeState: _isExpanded
  ? CrossFadeState.showSecond
  : CrossFadeState.showFirst,
duration: const Duration(milliseconds: 300),
),
],
),
),
);
}
}

```

